Hack The World(s)

Azéma Tanguy, Kheng Augustin, Le Pendeven Théo

June 19, 2026

EN PARTENARIAT AVEC

Agenda

Data & Project Tracks

Two kinds of data: the **datasets we provide** or **data you generate yourself**.
Pick a track below — or use one as a springboard for your own idea.

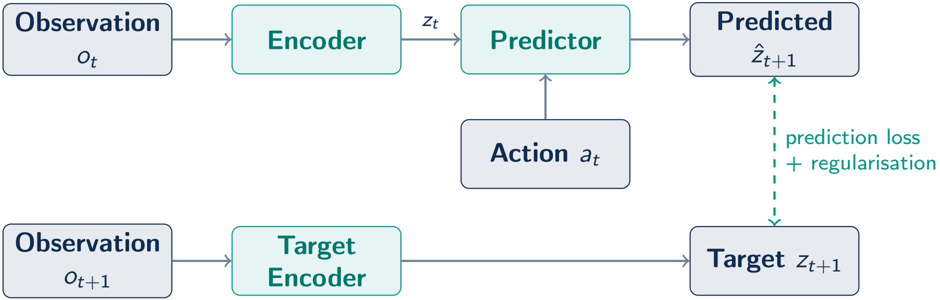
Group A — A new modality

1 Neuro - EEG
2 Audio / speech representation
3 Wearable biosignals (ECG / IMU / PPG)
4 Physical fields — *The Well*
5 Weather forecasting
6 Financial time series
7 Anomaly & predictive maintenance
8 3D point clouds / LiDAR

Group B — Vision & robotics

9 Learned cost / value for planning
10 Hierarchical / multi-timescale world model
11 Stress-test under factors of variation
12 Does intuitive physics emerge?

The Model — A JEPA Baseline



What we provide

A working **JEPA** that predicts *in representation space*, with regularisation losses to prevent collapse. Three tracks: **image**, **video**, **action-conditioned video**.

Pipeline — one engine: swappable modules

One generic, config-driven JEPA engine in `eb_jepa/`; each example just wires swappable core modules.

Core engine — `eb_jepa/`

- `jepa.py` — `JEPA`/`JEPAProbe`, unified `unroll()`
- `architectures.py` — encoders / predictors (ResNet, Impala, GRU)
- `losses.py` — predictive JEPA & anti-collapse (VICReg / VC / IDM)
- `planning.py` — CEM / MPPI planners & objectives

Around it — data & assembly

- `datasets/<env>/` — loaders, dispatched by `env_name`
- `examples/<x>/` — thin scripts: `main.py` · `eval.py` · `config.yaml` · README
- `launch_sbatch.py` — one launcher (`-example`, submitit)
- `tests/` lock the swappable-module contracts

Pipeline — examples to reuse, tracks to complete, and above all: explore

Reuse a complete example, complete a track baseline by filling a tiny `#TODO` surface — then explore.

Reuse — complete examples

- `image_jepa` — CIFAR-10 SSL, linear probe ~91%
- `video_jepa` — Moving-MNIST latent prediction
- `ac_video_jepa/two_rooms` — world model + planning (~97%)
- *explore: **the maze** — A^*-free hierarchical navigation*

Complete — 6 tracks (baseline + `#TODO`)

- audio · EEG · point cloud · FinTime · LTSF · Gray-Scott
- **provided & runnable**: loader + augment + train loop + probe harness
- **you fill 3 `#TODO` stubs**: `build_encoder` · `build_ssl/build_jepa` · `probe`

"Start from a baseline, then make it yours": runnable harness → fill the stubs → your baseline → explore.

What We Expect From You

Start from a baseline example, then make it yours.

01 A working model

Train a JEPA world model on the data of your choice.

- Keep it reproducible (`config.yaml`)
- Save at least one stable checkpoint

02 10-minute presentation

Walk us through your approach, experiments, and findings.

- Explain your architectural choices
- Show what the model learned

The Gold Standard

What makes a great submission?

- **Clear problem framing**
- **Quantitative results** vs. baseline
- **A genuine insight** or finding
- **Honest discussion** of limits
- **At least one ablation** study

✓

What Your Presentation Should Cover

Your 10-minute presentation should address the points below.

Data $\rightarrow$ **Architecture** $\rightarrow$ **Training** $\rightarrow$ **Inference** $\rightarrow$ **Evaluation**

(plus optional **Bonuses** to go further)

Explain the data you trained on.

1 Where is it from?

- **Generated** by you
- **Chosen** from the provided datasets
- **Found elsewhere** (cite the source)

Modality, size, difficulty?

2 What does it look like?

Show it — stats, samples, a projection (PCA):

3 How did you prepare it?

- Normalisation
- Encoding / decoding target
- Data augmentation?

Any preprocessing worth noting?

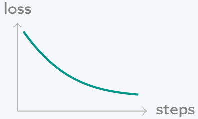
Describe your model and how it was trained.

The model

- **Unchanged?** A quick JEPA recap
- **Modified?** Why, what, intuition, papers
- Number of parameters & modules

Loss & training dynamics

- Which (sub)losses? how balanced?
- Representation collapse — observed?
- Regularisation & stability



training curve

Training

How did you train, and how did you iterate efficiently?

Setup

- Batch size, optimiser
- Learning-rate scheduler
- Number of epochs

Comparing without full training

- A **proxy metric** to rank runs early
- A strategy to compare *without* a full run

e.g. scaling laws, ablations, . . .

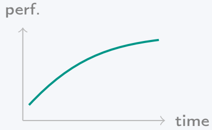
Inference

How do you use the trained model?

Inference strategy

- Method → Mode 1 / Mode 2, MCTS, actor, ... ?
- Performance vs. inference time
- Any inference tricks?

Performance vs. time



more compute, better rollout?

Evaluation

How good is it, and how do you know?

Robustness

- Ablations
- Scaling laws
- Seed & training stability

Performance

- Dataset performance
- Compare to a baseline

Reach

- Application to real-life situations
- A step towards AGI?

Bonuses

Optional — ways to stand out.

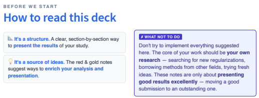
Rewarded

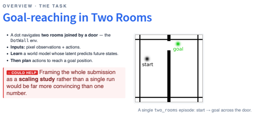
- High performance
- Ablations Hyperparameter tuning
- Extensive training-stability analysis — esp. visualising representation collapse

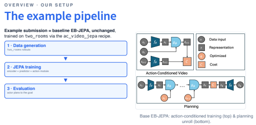
Going further

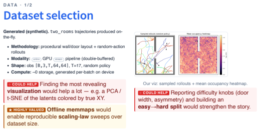
- Handmade (non-synthetic) dataset
- Method that scales to other datasets of the field (e.g. time series)
- Hierarchical world models

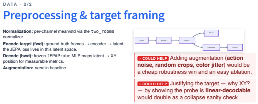
Two Rooms Example[1]

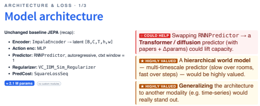


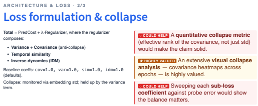


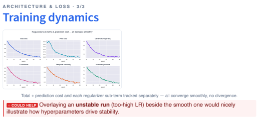










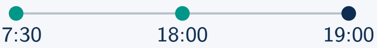


[0]Found it at `hackathon_guide/two_rooms_example_slides.pdf`

Timeline & Judging

Schedule

- 17:30 Code submission
- 18:00 Slides submission & pre-jury
- 19:00 Final jury *(qualified teams)*



Judging Criteria

- **Technical quality**
  Robustness, code cleanliness & architecture
- **Results**
  Performance, metrics & meaningful comparisons
- **Presentation**
  Clarity, storytelling & visual aids
- **Originality**
  Ablations, novel insights & going the extra mile

Now go build your world.

Good luck, and have fun!